

# Multi-Agent RAG

Architecture and technical guide for the supervisor multi-agent Retrieval-Augmented Generation pipeline — built on top of the project's production scaffolding (FastAPI, checkpointer, observability, Docker/K8s) and selectable via a single `AGENT_MODE` flag.

Document type: Architecture & Guide

Generated: 2026-06-05

74 tests · ruff · mypy clean

Verified live (Azure OpenAI)

## Contents

[1. Overview](#)

[2. Why Multi-Agent RAG](#)

[3. The Agents & Graph](#)

[4. Advanced Retrieval Pipeline](#)

[5. Component Reference](#)

[6. Configuration & Tuning](#)

[7. Run & Use It](#)

[8. Deployment & Scaling](#)

[9. Quality & Tests](#)

[10. Roadmap & Status](#)

# 1. Overview

The project now exposes **two interchangeable agent services** behind one interface ( `AgentService` ) and one selector ( `AGENT_MODE` ):

| Mode                             | What it is                                                                                                 | Best for                                                  |
|----------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>single</code><br>(default) | ReAct tool-using agent (web search · KB · calculator)                                                      | General Q&A, tool use, calculations                       |
| <code>multiagent</code>          | Supervisor RAG pipeline: retrieval → synthesis → critic, with hybrid search + reranking + grounding checks | Document-grounded answers with citations and verification |

Both share the same FastAPI surface, authentication, rate limiting, checkpointer, metrics, logging, and deployment artifacts. Switching is a config change — no code change.

## 2. Why Multi-Agent RAG

### Separation of duties

A supervisor routes, a retrieval agent gathers evidence, a synthesis agent writes, a critic verifies. Each is small, testable, and independently improvable.

### Retrieval is the product

Most RAG failures are retrieval failures. Hybrid search + reranking + multi-query expansion materially raise recall and precision over plain vector search.

### Grounding & trust

The critic checks that every claim is supported by retrieved context and can trigger a bounded re-retrieval — reducing hallucination and enabling citations.

### Bounded cost

Handoffs are explicit and revision rounds are capped ( `MAX_REVISIONS` ), so agent "chatter" can't blow up latency or tokens.

### 3. The Agents & Graph

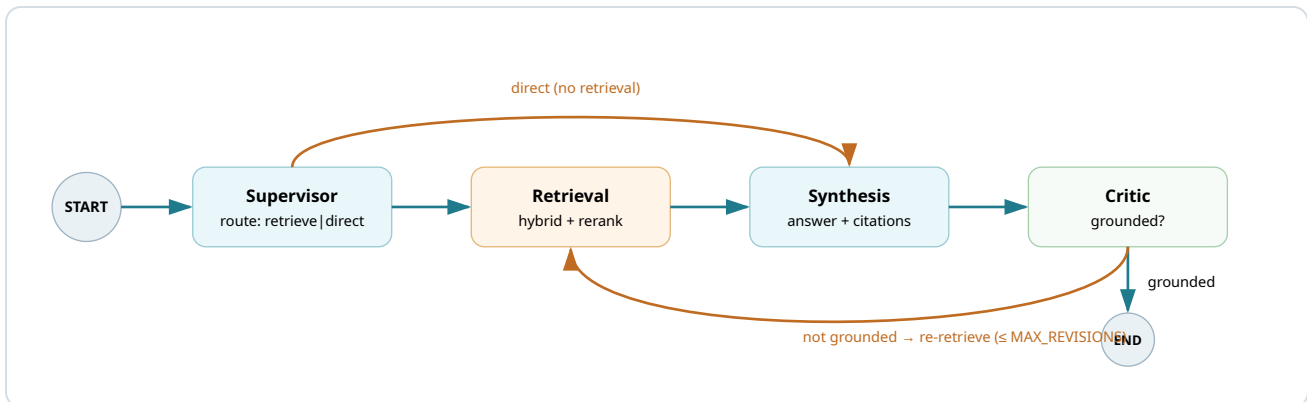


Figure 1 — Implemented as a LangGraph `StateGraph` with conditional edges. Nodes call the LLM directly; the retriever and query expander are injected (so the graph is unit-testable with fakes).

| Agent      | Responsibility                                                                                                    | Output to state                                                              |
|------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Supervisor | Classify whether retrieval is needed ( <code>RETRIEVE</code> / <code>DIRECT</code> ); reset per-run scratch state | <code>route</code>                                                           |
| Retrieval  | Expand query → hybrid search per query → fuse + cap context                                                       | <code>queries</code> ,<br><code>documents</code>                             |
| Synthesis  | Draft a grounded answer with inline <code>[n]</code> citations (or a direct answer)                               | <code>draft</code> , <code>answer</code>                                     |
| Critic     | Verify grounding; emit <code>GROUNDING</code> / <code>NOT_GROUNDING</code> + feedback; bounded loop               | <code>grounded</code> ,<br><code>critique</code> ,<br><code>revisions</code> |

## 4. Advanced Retrieval Pipeline

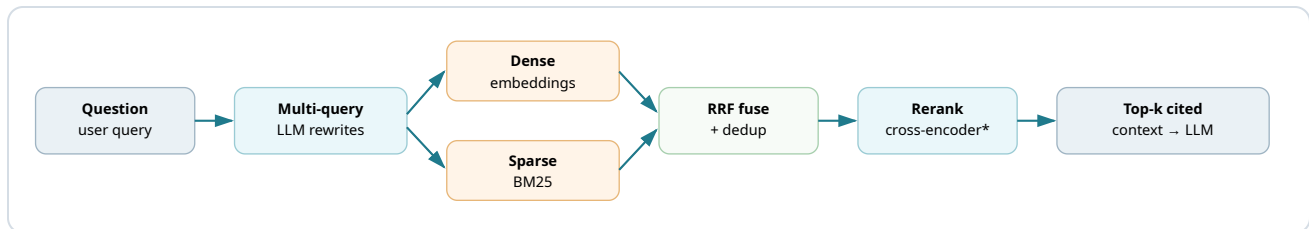


Figure 2 — \*Reranking is optional ( `RERANK_ENABLED` ); it downloads a cross-encoder on first use. Without it, the fused top-k is used directly.

- **Multi-query expansion** — the LLM rewrites the question into several phrasings; results are unioned (recall ↑). Degrades gracefully to the original query if the LLM is unavailable.
- **Hybrid search** — dense (vector embeddings) + sparse (BM25) retrieve in parallel; complementary strengths (semantics vs exact terms).
- **Reciprocal Rank Fusion** — combines the two rankings by rank position (robust to score scales), de-duplicating by source+content.
- **Reranking** — an optional cross-encoder scores each (query, chunk) pair jointly for a precision boost before the top-k is sent to the model.
- **Critic loop** — if the answer isn't grounded, the critic's feedback seeds a focused re-retrieval, bounded by `MAX_REVISIONS` .

## 5. Component Reference

```
src/agentic_research_agent/  
├─ rag/                                # advanced retrieval building blocks  
│   ├── retriever.py                  # HybridRetriever (dense + BM25 + RRF), RetrievedDoc  
│   ├── reranker.py                   # Reranker protocol + CrossEncoderReranker (lazy)  
│   └─ query_transform.py             # QueryTransformer (multi-query, graceful fallback)  
├─ multiagent/                         # the supervisor RAG graph  
│   ├── state.py                     # MultiAgentState, RetrievedChunk  
│   ├── prompts.py                   # supervisor / synthesis / critic prompts  
│   ├── graph.py                     # build_multiagent_graph (StateGraph + routing)  
│   └─ service.py                    # MultiAgentRAG facade (ask / stream / health)  
├─ agents/base.py                     # AgentService protocol (shared interface)  
└─ service_factory.py                 # build_agent_service() → single | multiagent
```

The retriever is decoupled from any store: it takes a `dense_search` callable plus an in-memory corpus for BM25, so fusion is unit-testable without embeddings.

`build_hybrid_retriever()` adapts the existing `KnowledgeBase` (Chroma) into those inputs.

## 6. Configuration & Tuning

| Variable                                                        | Default                                   | Effect                                            |
|-----------------------------------------------------------------|-------------------------------------------|---------------------------------------------------|
| <code>AGENT_MODE</code>                                         | <code>single</code>                       | <code>multiagent</code> enables the RAG pipeline  |
| <code>RETRIEVER_TOP_K</code>                                    | <code>4</code>                            | Chunks fed to the model per query                 |
| <code>RETRIEVER_FETCH_K</code>                                  | <code>10</code>                           | Candidates per retriever before fusion            |
| <code>MULTIQUERY_ENABLED</code> / <code>MULTIQUERY_COUNT</code> | <code>true</code> / <code>3</code>        | Query expansion breadth (recall vs cost)          |
| <code>RERANK_ENABLED</code> / <code>RERANK_MODEL</code>         | <code>false</code> / MiniLM cross-encoder | Precision boost (adds latency + a model download) |
| <code>MAX_REVISIONS</code>                                      | <code>1</code>                            | Critic re-retrieval rounds (quality vs latency)   |
| <code>CHUNK_SIZE</code> / <code>CHUNK_OVERLAP</code>            | <code>1000</code> / <code>150</code>      | Ingestion chunking (re-ingest after changes)      |

**Tuning order:** (1) get chunking right, (2) enable hybrid + multi-query (on by default), (3) turn on reranking for precision, (4) raise `MAX_REVISIONS` only if grounding is still weak — each step trades latency/cost for quality.

## 7. Run & Use It

```
# 1. install + configure + ingest (same as single mode)
uv sync
cp .env.example .env          # set a provider key
uv run research-agent ingest

# 2. select multi-agent RAG
export AGENT_MODE=multiagent

# CLI
uv run research-agent ask "What is RAG? Cite sources."
uv run research-agent chat

# HTTP API (same endpoints, auth, metrics)
uv run research-agent-api
curl -X POST localhost:8000/v1/ask -H "X-API-Key: $API_KEYS" \
  -H "Content-Type: application/json" \
  -d '{"question":"Explain the ReAct pattern.","thread_id":"demo"}'
```

The streaming endpoint surfaces each agent step:

```
[supervisor] route=retrieve
[retrieval] 4 chunks
[synthesis] drafted answer
[critic] grounded=True
<final answer with [n] citations>
```

## 8. Deployment & Scaling

Multi-agent mode runs on the **same** production stack as single mode — FastAPI + Uvicorn/Gunicorn, durable checkpointer, Docker/Compose/Kubernetes, Prometheus metrics, health probes. See the [Production-Grade Guide](#) for the step-by-step deploy and base scaling. RAG-specific considerations:

| Concern                    | Guidance                                                                                                                                                                                                        |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Latency / cost per request | More LLM calls per answer (supervisor + synthesis + critic + multi-query). Use a fast model for supervisor/critic; cap <code>MULTIQUERY_COUNT</code> and <code>MAX_REVISIONS</code> ; cache repeated questions. |
| Vector store               | Local Chroma is fine for one node; for multi-replica move to a <b>shared vector DB</b> (pgvector/Qdrant). The <code>build_hybrid_retriever</code> seam isolates this swap.                                      |
| BM25 index                 | Built in-process from the corpus at startup. For very large corpora, move sparse retrieval into the vector DB (e.g. Qdrant/Elasticsearch hybrid) instead of in-memory BM25.                                     |
| Reranker                   | CPU-heavy cross-encoder. Run it as a dedicated service or use a hosted reranker at scale; keep it off the hot path when latency-critical.                                                                       |
| Ingestion                  | Keep as an out-of-band job that writes to the shared vector DB; the serving path only reads.                                                                                                                    |
| Scale-out                  | The API tier stays stateless → scale horizontally with HPA. Set <code>maxReplicas</code> below the LLM provider's concurrency ceiling.                                                                          |

## 9. Quality & Tests

The suite is fully offline (fake LLMs + fake retrievers — no API key, no network, no model download):

- **Retriever** — RRF fusion, BM25 lexical-overlap matching, de-dup, reranker application.
- **Query transform** — passthrough without an LLM, parsing of rewrites, graceful failure.
- **Multi-agent graph** — retrieve path, direct path, critic-triggered revision, and revision-budget enforcement.
- **API** — auth, rate limit, input cap, health, metrics (mode-agnostic via the shared protocol).

```
uv run pytest          # 74 tests
uv run ruff check .    · uv run ruff format --check .    · uv run mypy    # clean
```

Verified live against Azure OpenAI: `supervisor → retrieval(4 chunks) → synthesis → critic(grounded) → cited answer`.

## 10. Roadmap & Status

| Phase                    | Scope                                                                                                     | Status    |
|--------------------------|-----------------------------------------------------------------------------------------------------------|-----------|
| 0 — Foundation           | Reuse API/config/checkpointer/observability/deploy                                                        | Done      |
| 1 — Advanced RAG         | Hybrid retrieval + RRF + multi-query + optional rerank + cited generation                                 | Done      |
| 2 — Multi-agent          | Supervisor + retrieval + synthesis + critic, bounded revisions, streaming                                 | Done      |
| 3 — Production hardening | Auth, rate limit, durable state, metrics, tracing, containers, CI                                         | Inherited |
| 4 — Scale & operate      | Shared vector DB (pgvector/Qdrant), semantic cache, eval gate in CI, model routing/fallback, online evals | Next      |

**Recommended next steps:** (1) swap Chroma → pgvector/Qdrant behind the retriever seam; (2) add a RAGAS/LangSmith *eval gate* to CI (retrieval recall + answer faithfulness); (3) add a Redis semantic cache and a cheap-model supervisor to control cost.